

Design System Specification

Christmas Tracker iOS Application

Version: 1.0
Date: February 3, 2026
Status: Approved

Overview

This document defines the complete design system for the Christmas Tracker iOS application, based on the official theme specification. It includes color palettes, typography, spacing, component patterns, and accessibility requirements optimized for iOS 26 and SwiftUI.

Design Principles:

- **Warm & Organized:** Calm, structured, subtly festive year-round
- **iOS-Native:** Follow Human Interface Guidelines
- **Accessible:** WCAG AA minimum, full VoiceOver support
- **Consistent:** Predictable patterns, reusable components

1. Color System

1.1 Asset Catalog Structure

All colors are defined in `Assets.xcassets/Colors/` with both light and dark mode variants.

```
Colors/  
├── Primary/  
│   └── primary.colorset  
├── Accent/  
│   └── accent.colorset  
├── Highlight/  
│   └── highlight.colorset  
├── Background/  
│   ├── background-primary.colorset  
│   └── background-secondary.colorset  
├── Surface/  
│   └── surface.colorset  
├── Text/  
│   ├── text-primary.colorset  
│   └── text-muted.colorset  
├── Border/  
│   └── border.colorset  
└── Status/  
    ├── available.colorset  
    ├── purchased.colorset  
    ├── partial.colorset  
    ├── shipped.colorset  
    ├── wrapped.colorset  
    └── gifted.colorset
```

1.2 Brand Colors

Primary (Evergreen)

Light Mode: #1F4D3A

Dark Mode: #4FA58A

Usage: Primary buttons, headers, navigation, emphasis, active states

```
// primary.colorset/Contents.json
{
  "colors": [
    {
      "color": {
        "color-space": "srgb",
        "components": {
          "red": "0.122",
          "green": "0.302",
          "blue": "0.227",
          "alpha": "1.000"
        }
      },
      "idiom": "universal"
    },
    {
      "appearances": [
        {
          "appearance": "luminosity",
          "value": "dark"
        }
      ],
      "color": {
        "color-space": "srgb",
        "components": {
          "red": "0.310",
          "green": "0.647",
          "blue": "0.541",
          "alpha": "1.000"
        }
      },
      "idiom": "universal"
    }
  ],
  "info": {
    "author": "xcode",
    "version": 1
  }
}
```

Accent (Cranberry)

Light Mode: #B23A48

Dark Mode: #E06A75

Usage: Accents, alerts, destructive actions, highlights

Highlight (Warm Gold)

Light Mode: #D6B25E

Dark Mode: #E4C76A

Usage: Completion states, success indicators, emphasis, warnings

1.3 Neutral Colors

Background - Primary

Light Mode: #FAF9F7
Dark Mode: #0F1A17
Usage: Main app background

Background - Secondary

Light Mode: #F2EFEA
Dark Mode: #162622
Usage: Sections, panels, dashboard zones

Surface / Card

Light Mode: #FFFFFF
Dark Mode: #1E2E2A
Usage: Cards, modals, list containers

Border / Divider

Light Mode: #E2E2E2
Dark Mode: #2C3F3A
Usage: Borders, separators, table lines

1.4 Text Colors

Text - Primary

Light Mode: #1F2933
Dark Mode: #E6CECA
Usage: Main text, headings, body copy

Text - Muted

Light Mode: #6B7280
Dark Mode: #9AA5A1
Usage: Secondary text, metadata, timestamps

1.5 Status Colors

Available

Light Mode: #1F4D3A
Dark Mode: #4FA58A
Meaning: Ready to be purchased

Purchased

Light Mode: #2F7F6D
Dark Mode: #63BFA8
Meaning: Purchased successfully

Partial

Light Mode: #D6B25E
Dark Mode: #E4C76A
Meaning: Partially fulfilled

Shipped

Light Mode: #3A7CA5

Dark Mode: #6FAED6

Meaning: In transit

Wrapped

Light Mode: #B23A48

Dark Mode: #E06A75

Meaning: Wrapped and ready

Gifted

Light Mode: #C9A227

Dark Mode: #F0D97C

Meaning: Delivered / completed

1.6 Budget State Colors

Under Budget (Safe)

Light Mode: #1F4D3A

Dark Mode: #4FA58A

Near Limit (Warning)

Light Mode: #D6B25E

Dark Mode: #E4C76A

Over Budget (Alert)

Light Mode: #B23A48

Dark Mode: #E06A75

1.7 Swift Color Extensions

```
// DesignSystem/Colors/AppColors.swift

import SwiftUI

extension Color {
    // MARK: – Brand Colors

    /// Primary brand color (Evergreen)
    /// Light: #1F4D3A, Dark: #4FA58A
    static let primaryColor = Color("primary")

    /// Accent color (Cranberry)
    /// Light: #B23A48, Dark: #E06A75
    static let accentColor = Color("accent")

    /// Highlight color (Warm Gold)
    /// Light: #D6B25E, Dark: #E4C76A
    static let highlightColor = Color("highlight")

    // MARK: – Background Colors

    /// Primary background color
    /// Light: #FAF9F7, Dark: #0F1A17
    static let backgroundPrimaryColor = Color("background-primary")
```

```

/// Secondary background color
/// Light: #F2EFEA, Dark: #162622
static let backgroundSecondaryColor = Color("background-secondary")

/// Surface color for cards/modals
/// Light: #FFFFFF, Dark: #1E2E2A
static let surfaceColor = Color("surface")

// MARK: - Text Colors

/// Primary text color
/// Light: #1F2933, Dark: #E6ECEA
static let textPrimaryColor = Color("text-primary")

/// Muted text color
/// Light: #6B7280, Dark: #9AA5A1
static let textMutedColor = Color("text-muted")

// MARK: - Border Colors

/// Border and divider color
/// Light: #E2E2E2, Dark: #2C3F3A
static let borderColor = Color("border")

// MARK: - Status Colors

static let statusAvailableColor = Color("available")
static let statusPurchasedColor = Color("purchased")
static let statusPartialColor = Color("partial")
static let statusShippedColor = Color("shipped")
static let statusWrappedColor = Color("wrapped")
static let statusGiftedColor = Color("gifted")

// MARK: - Budget State Colors

static let budgetSafeColor = Color("budget-safe")
static let budgetWarningColor = Color("budget-warning")
static let budgetOverColor = Color("budget-over")
}

// MARK: - Status Color Helper

extension GiftItem {
    var statusColor: Color {
        if isPurchased {
            return .statusPurchasedColor
        }
        if isOffList {
            return .statusWrappedColor
        }
        return .statusAvailableColor
    }
}

```

2. Typography System

2.1 Font Family

Primary Font: **Inter** (system default with SF Pro fallback)
Accent Font: **Playfair Display** (limited use)

2.2 Type Scale

Following iOS Dynamic Type sizing with custom scale:

Style	Size (pts)	Weight	Usage
Large Title	34	Bold (700)	Screen titles
Title 1	28	Semibold (600)	Section headers
Title 2	22	Semibold (600)	Card titles
Title 3	20	Semibold (600)	List headers
Headline	17	Semibold (600)	Emphasis
Body	17	Regular (400)	Body text
Callout	16	Regular (400)	Secondary text
Subheadline	15	Regular (400)	Metadata
Footnote	13	Regular (400)	Small text
Caption 1	12	Regular (400)	Timestamps
Caption 2	11	Regular (400)	Fine print

2.3 Font Weights

- **Regular:** 400 (body text, labels)
- **Medium:** 500 (metadata, secondary emphasis)
- **Semibold:** 600 (buttons, headers)
- **Bold:** 700 (large titles, strong emphasis)

2.4 SwiftUI Typography Extensions

```
// DesignSystem/Typography/AppTypography.swift

import SwiftUI

extension Font {
    // MARK: - Primary Font (Inter/SF Pro)

    static let appLargeTitle = Font.system(size: 34, weight: .bold, design: .default)
    static let appTitle1 = Font.system(size: 28, weight: .semibold, design: .default)
    static let appTitle2 = Font.system(size: 22, weight: .semibold, design: .default)
    static let appTitle3 = Font.system(size: 20, weight: .semibold, design: .default)
    static let appHeadline = Font.system(size: 17, weight: .semibold, design: .default)
    static let appBody = Font.system(size: 17, weight: .regular, design: .default)
    static let appCallout = Font.system(size: 16, weight: .regular, design: .default)
    static let appSubheadline = Font.system(size: 15, weight: .regular, design: .default)
    static let appFootnote = Font.system(size: 13, weight: .regular, design: .default)
    static let appCaption1 = Font.system(size: 12, weight: .regular, design: .default)
    static let appCaption2 = Font.system(size: 11, weight: .regular, design: .default)

    // MARK: - Accent Font (Playfair Display)
    // Used sparingly for app name, seasonal headers

    static let appAccentLarge = Font.custom("PlayfairDisplay-Bold", size: 34)
```

```

    static let appAccentTitle = Font.custom("PlayfairDisplay-Bold", size: 28)
}

// MARK: - Text Style Modifiers

extension Text {
    func appLargeTitle() -> Text {
        self.font(.appLargeTitle)
    }

    func appTitle1() -> Text {
        self.font(.appTitle1)
    }

    func appTitle2() -> Text {
        self.font(.appTitle2)
    }

    func appBody() -> Text {
        self.font(.appBody)
    }

    func appCaption() -> Text {
        self.font(.appCaption1).foregroundColor(.textMutedColor)
    }
}

```

3. Spacing System

3.1 Spacing Scale

Following 4pt base unit:

Token	Value	Usage
<code>xs</code>	4pt	Icon padding, tight spacing
<code>sm</code>	8pt	Compact layouts, list items
<code>md</code>	12pt	Standard element spacing
<code>lg</code>	16pt	Section padding, card padding
<code>xl</code>	24pt	Large gaps, screen margins
<code>2xl</code>	32pt	Major section separation
<code>3xl</code>	48pt	Screen top/bottom padding

3.2 SwiftUI Spacing Constants

```

// DesignSystem/Spacing/AppSpacing.swift

import SwiftUI

enum AppSpacing {
    static let xs: CGFloat = 4
    static let sm: CGFloat = 8
    static let md: CGFloat = 12
    static let lg: CGFloat = 16
    static let xl: CGFloat = 24
}

```

```

static let xxl: CGFloat = 32
static let xxxl: CGFloat = 48
}

extension EdgeInsets {
    static let appCardPadding = EdgeInsets(
        top: AppSpacing.lg,
        leading: AppSpacing.lg,
        bottom: AppSpacing.lg,
        trailing: AppSpacing.lg
    )

    static let appScreenPadding = EdgeInsets(
        top: AppSpacing.xl,
        leading: AppSpacing.lg,
        bottom: AppSpacing.xl,
        trailing: AppSpacing.lg
    )
}

```

4. Component Library

4.1 Buttons

Primary Button

```

// DesignSystem/Components/Buttons/PrimaryButton.swift

import SwiftUI

struct PrimaryButton: View {
    let title: String
    let action: () -> Void
    var isLoading: Bool = false
    var isDisabled: Bool = false

    var body: some View {
        Button(action: action) {
            HStack(spacing: AppSpacing.sm) {
                if isLoading {
                    ProgressView()
                        .tint(.white)
                } else {
                    Text(title)
                        .font(.appHeadline)
                }
            }
            .frame(maxWidth: .infinity)
            .frame(height: 50)
            .foregroundColor(.white)
            .background(isDisabled ? Color.textMutedColor : Color.primaryColor)
            .cornerRadius(12)
        }
        .disabled(isDisabled || isLoading)
    }
}

#Preview {
    VStack(spacing: AppSpacing.lg) {

```

```

        PrimaryButton(title: "Log In") { }
        PrimaryButton(title: "Loading", isLoading: true) { }
        PrimaryButton(title: "Disabled", isDisabled: true) { }
    }
    .padding()
}

```

Secondary Button

```

struct SecondaryButton: View {
    let title: String
    let action: () -> Void
    var isDisabled: Bool = false

    var body: some View {
        Button(action: action) {
            Text(title)
                .font(.appHeadline)
                .frame(maxWidth: .infinity)
                .frame(height: 50)
                .foregroundColor(isDisabled ? .textMutedColor : .primaryColor)
                .background(Color.surfaceColor)
                .overlay(
                    RoundedRectangle(cornerRadius: 12)
                        .stroke(isDisabled ? Color.borderColor : Color.primaryColor, lineWidth: 2)
                )
        }
        .disabled(isDisabled)
    }
}

```

Destructive Button

```

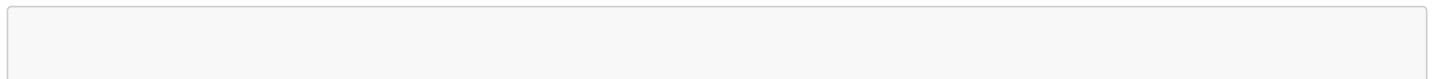
struct DestructiveButton: View {
    let title: String
    let action: () -> Void

    var body: some View {
        Button(action: action) {
            Text(title)
                .font(.appHeadline)
                .frame(maxWidth: .infinity)
                .frame(height: 50)
                .foregroundColor(.white)
                .background(Color.accentColor)
                .cornerRadius(12)
        }
    }
}

```

4.2 Cards

List Card



```

// DesignSystem/Components/Cards/ListCard.swift

struct ListCard: View {
    let list: GiftList
    let onTap: () -> Void

    var body: some View {
        Button(action: onTap) {
            VStack(alignment: .leading, spacing: AppSpacing.md) {
                // Header
                HStack {
                    Text(list.name)
                        .font(.appTitle3)
                        .foregroundColor(.textPrimaryColor)

                    Spacer()

                    Image(systemName: "chevron.right")
                        .foregroundColor(.textMutedColor)
                }

                // Stats
                HStack(spacing: AppSpacing.lg) {
                    StatBadge(
                        icon: "gift",
                        value: "\\(list.totalItems)",
                        label: "Items"
                    )

                    StatBadge(
                        icon: "checkmark.circle",
                        value: "\\(list.purchasedItems)",
                        label: "Purchased"
                    )

                    StatBadge(
                        icon: "dollarsign.circle",
                        value: "\\$\\(list.budget, specifier: "%.0f")",
                        label: "Budget"
                    )
                }
            }
            .padding(EdgeInsets.appCardPadding)
            .background(Color.surfaceColor)
            .cornerRadius(12)
            .shadow(color: Color.black.opacity(0.05), radius: 4, x: 0, y: 2)
        }
        .buttonStyle(.plain)
    }
}

struct StatBadge: View {
    let icon: String
    let value: String
    let label: String

    var body: some View {
        VStack(alignment: .leading, spacing: AppSpacing.xs) {
            HStack(spacing: AppSpacing.xs) {
                Image(systemName: icon)
                    .foregroundColor(.primaryColor)
                    .font(.appFootnote)
            }
        }
    }
}

```

```

        Text(value)
            .font(.appHeadline)
            .foregroundColor(.textPrimaryColor)
    }

    Text(label)
        .font(.appCaption1)
        .foregroundColor(.textMutedColor)
    }
}
}

```

4.3 Input Fields

Text Field Style

```

// DesignSystem/Components/Inputs/TextFieldStyle.swift

struct AppTextFieldStyle: TextFieldStyle {
    func _body(configuration: TextField<Self._Label>) -> some View {
        configuration
            .padding(AppSpacing.md)
            .font(.appBody)
            .background(Color.surfaceColor)
            .overlay(
                RoundedRectangle(cornerRadius: 8)
                    .stroke(Color.borderColor, lineWidth: 1)
            )
    }
}

extension View {
    func appTextFieldStyle() -> some View {
        self.textFieldStyle(AppTextFieldStyle())
    }
}

// Usage
TextField("Email", text: $email)
    .appTextFieldStyle()

```

4.4 Loading States

Loading View

```

struct LoadingView: View {
    var message: String = "Loading..."

    var body: some View {
        VStack(spacing: AppSpacing.lg) {
            ProgressView()
                .tint(.primaryColor)

            Text(message)
                .font(.appBody)
                .foregroundColor(.textMutedColor)
        }
    }
}

```

```

    }
}

```

Skeleton View

```

struct SkeletonView: View {
    @State private var animate = false

    var body: some View {
        RoundedRectangle(cornerRadius: 8)
            .fill(
                LinearGradient(
                    colors: [
                        Color.backgroundSecondaryColor,
                        Color.backgroundSecondaryColor.opacity(0.7),
                        Color.backgroundSecondaryColor
                    ],
                    startPoint: .leading,
                    endPoint: .trailing
                )
            )
            .frame(height: 80)
            .onAppear {
                withAnimation(.linear(duration: 1.5).repeatForever(autoreverses: false)) {
                    animate.toggle()
                }
            }
    }
}

```

4.5 Empty States

```

struct EmptyStateView: View {
    let icon: String
    let title: String
    let message: String
    var actionTitle: String?
    var action: (() -> Void)?

    var body: some View {
        VStack(spacing: AppSpacing.lg) {
            Image(systemName: icon)
                .font(.system(size: 64))
                .foregroundColor(.textMutedColor)

            VStack(spacing: AppSpacing.sm) {
                Text(title)
                    .font(.appTitle2)
                    .foregroundColor(.textPrimaryColor)

                Text(message)
                    .font(.appBody)
                    .foregroundColor(.textMutedColor)
                    .multilineTextAlignment(.center)
            }

            if let actionTitle = actionTitle, let action = action {
                PrimaryButton(title: actionTitle, action: action)
            }
        }
    }
}

```

```

        .frame(maxWidth: 200)
    }
}
.padding(EdgeInsets.appScreenPadding)
}
}

// Usage
EmptyStateView(
    icon: "gift.fill",
    title: "No Lists Yet",
    message: "Create your first gift list to get started",
    actionTitle: "Create List",
    action: { }
)

```

5. SF Symbols Mapping

5.1 Icon System

All icons use SF Symbols for consistency and proper scaling.

Feature	Icon	Symbol Name
Navigation		
Dashboard		house.fill
Stats		chart.bar.fill
Profile		person.fill
Actions		
Add		plus
Edit		pencil
Delete		trash
Share		square.and.arrow.up
Filter		line.3.horizontal.decrease
Status		
Available		circle
Purchased		checkmark.circle.fill
Shipped		shippingbox.fill
Wrapped		gift.fill
Lists		
List		list.bullet
Owned		crown.fill
Member		person.2.fill
Invitation		envelope.fill
Budget		

Feature	Icon	Symbol Name
Budget		<code>dollarsign.circle.fill</code>
Safe		<code>checkmark.circle.fill</code>
Warning		<code>exclamationmark.triangle.fill</code>
Over		<code>xmark.circle.fill</code>

5.2 Icon Extension

```
// DesignSystem/Icons/AppIcons.swift

import SwiftUI

enum AppIcon {
    // Navigation
    static let dashboard = "house.fill"
    static let stats = "chart.bar.fill"
    static let profile = "person.fill"

    // Actions
    static let add = "plus"
    static let edit = "pencil"
    static let delete = "trash"
    static let share = "square.and.arrow.up"
    static let filter = "line.3.horizontal.decrease"

    // Status
    static let available = "circle"
    static let purchased = "checkmark.circle.fill"
    static let shipped = "shippingbox.fill"
    static let wrapped = "gift.fill"
    static let gifted = "checkmark.seal.fill"

    // Lists
    static let list = "list.bullet"
    static let owned = "crown.fill"
    static let member = "person.2.fill"
    static let invitation = "envelope.fill"

    // Budget
    static let budget = "dollarsign.circle.fill"
    static let safe = "checkmark.circle.fill"
    static let warning = "exclamationmark.triangle.fill"
    static let over = "xmark.circle.fill"
}
```

6. Layout & Grid System

6.1 Screen Margins

- **iPhone (Portrait):** 16pt left/right
- **iPhone (Landscape):** 32pt left/right
- **iPad:** 32pt left/right, adaptive up to 64pt on large screens

6.2 Grid Layout

iPhone: Single column
iPad: Multi-column grid (2-3 columns based on width)

```
struct AdaptiveGrid<Item: Identifiable, ItemView: View>: View {
    let items: [Item]
    let itemView: (Item) -> ItemView

    var body: some View {
        GeometryReader { geometry in
            let columns = geometry.size.width > 600 ? 2 : 1

            LazyVGrid(
                columns: Array(repeating: GridItem(.flexible()), spacing: AppSpacing.lg), count:
columns),
                spacing: AppSpacing.lg
            ) {
                ForEach(items) { item in
                    itemView(item)
                }
            }
        }
    }
}
```

7. Accessibility

7.1 Requirements

WCAG AA Compliance:

- **Contrast Ratio:** 4.5:1 minimum for normal text
- **Large Text:** 3:1 minimum for 18pt+ regular or 14pt+ bold
- **Touch Targets:** 44x44pt minimum

VoiceOver Support:

- All interactive elements labeled
- Meaningful hints for complex interactions
- Proper heading hierarchy

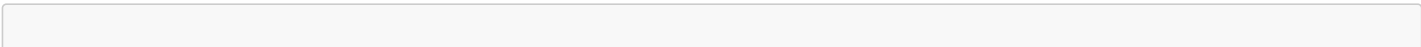
Dynamic Type:

- Support all text size categories
- Layout adapts to larger text
- No text clipping

7.2 Color Contrast Validation

Pair	Light Mode	Dark Mode	Passes
Text Primary / Background	13.6:1	12.8:1	✔ AAA
Text Muted / Background	4.9:1	4.6:1	✔ AA
Primary Button / White Text	5.8:1	3.2:1	✔ AA

7.3 Accessibility Modifiers



```

extension View {
    func appAccessibility(
        label: String,
        hint: String? = nil,
        trait: AccessibilityTraits? = nil
    ) -> some View {
        self
            .accessibilityLabel(label)
            .accessibilityHint(hint ?? "")
            .accessibilityAddTraits(trait ?? [])
    }
}

// Usage
Button("Delete") { }
    .appAccessibility(
        label: "Delete list",
        hint: "Permanently removes this list and all items",
        trait: .isButton
    )

```

8. Animation & Motion

8.1 Animation Tokens

```

enum AppAnimation {
    static let fast = Animation.easeInOut(duration: 0.2)
    static let normal = Animation.easeInOut(duration: 0.3)
    static let slow = Animation.easeInOut(duration: 0.5)

    static let spring = Animation.spring(response: 0.3, dampingFraction: 0.7)
}

```

8.2 Reduce Motion Support

```

@Environment(\.accessibilityReduceMotion) var reduceMotion

var animation: Animation {
    reduceMotion ? .none : .appSpring
}

```

9. Dark Mode Support

All colors defined in Asset Catalog automatically support dark mode. Additionally, users can control appearance mode independently of system settings.

9.1 AppearanceManager

```

// Core/Configuration/AppearanceManager.swift

import SwiftUI

@Observable

```

```

class AppearanceManager {
    enum AppearanceMode: String, Codable, CaseIterable {
        case light = "Light"
        case dark = "Dark"
        case system = "System"

        var icon: String {
            switch self {
            case .light: return "sun.max.fill"
            case .dark: return "moon.fill"
            case .system: return "circle.lefthalf.filled"
            }
        }
    }

    var currentMode: AppearanceMode {
        didSet {
            UserDefaults.standard.set(currentMode.rawValue, forKey: "appearanceMode")
            applyAppearance()
        }
    }

    init() {
        let savedMode = UserDefaults.standard.string(forKey: "appearanceMode")
        self.currentMode = AppearanceMode(rawValue: savedMode ?? "") ?? .system
    }

    func applyAppearance() {
        Task { @MainActor in
            guard let scene = UIApplication.shared.connectedScenes.first as? UIWindowScene,
                  let window = scene.windows.first else {
                return
            }

            switch currentMode {
            case .light:
                window.overrideUserInterfaceStyle = .light
            case .dark:
                window.overrideUserInterfaceStyle = .dark
            case .system:
                window.overrideUserInterfaceStyle = .unspecified
            }
        }
    }
}

// Environment key
private struct AppearanceManagerKey: EnvironmentKey {
    static let defaultValue = AppearanceManager()
}

extension EnvironmentValues {
    var appearanceManager: AppearanceManager {
        get { self[AppearanceManagerKey.self] }
        set { self[AppearanceManagerKey.self] = newValue }
    }
}

```

9.2 Appearance Settings UI

```
// Features/Profile/Views/AppearanceSettingsView.swift

struct AppearanceSettingsView: View {
    @Environment(\.appearanceManager) private var appearanceManager

    var body: some View {
        VStack(alignment: .leading, spacing: AppSpacing.lg) {
            Text("Appearance")
                .font(.appHeadline)
                .foregroundColor(.textPrimaryColor)

            Picker("Theme", selection: $appearanceManager.currentMode) {
                ForEach(AppearanceManager.AppearanceMode.allCases, id: \.self) { mode in
                    Label(mode.rawValue, systemImage: mode.icon)
                        .tag(mode)
                }
            }
                .pickerStyle(.segmented)

            Text("Choose how Christmas Tracker looks on your device")
                .font(.appCaption1)
                .foregroundColor(.textMutedColor)
        }
        .padding(EdgeInsets.appCardPadding)
        .background(Color.surfaceColor)
        .cornerRadius(12)
    }
}

#Preview {
    AppearanceSettingsView()
}
```

9.3 Testing Dark Mode

Manual Testing:

- Test all screens in both light and dark mode
- Verify contrast ratios meet WCAG AA standards
- Check for "pure black" (should use charcoal #0F1A17)
- Test with Dynamic Type at largest size

Xcode Preview:

```
#Preview("Light Mode") {
    DashboardView()
        .preferredColorScheme(.light)
}

#Preview("Dark Mode") {
    DashboardView()
        .preferredColorScheme(.dark)
}
```

10. Component Usage Examples

10.1 Login Screen

```

struct LoginView: View {
    @State private var email = ""
    @State private var password = ""

    var body: some View {
        VStack(spacing: AppSpacing.xl) {
            // Logo
            Text("Christmas Tracker")
                .font(.accentColorLarge)
                .foregroundColor(.primaryColor)

            // Inputs
            VStack(spacing: AppSpacing.lg) {
                TextField("Email", text: $email)
                    .appTextFieldStyle()
                    .textContentType(.emailAddress)
                    .autocapitalization(.none)

                SecureField("Password", text: $password)
                    .appTextFieldStyle()
                    .textContentType(.password)
            }

            // Action
            PrimaryButton(title: "Log In") {
                // Login action
            }

            Spacer()
        }
        .padding(EdgeInsets.appScreenPadding)
        .background(Color.backgroundPrimaryColor)
    }
}

```

11. Design Checklist

Before shipping any screen:

- ☐ All colors from Asset Catalog (no hardcoded hex)
- ☐ Typography uses defined Font extensions
- ☐ Spacing uses AppSpacing constants
- ☐ Buttons use component library
- ☐ VoiceOver labels on all interactive elements
- ☐ Tested with largest Dynamic Type size
- ☐ Tested in light and dark mode
- ☐ Touch targets minimum 44x44pt
- ☐ Loading states for async operations
- ☐ Empty states for zero-data screens
- ☐ Error states with retry actions

12. Resources

12.1 Design Files

- Figma: [Link to Figma file]

- Sketch: [Link to Sketch file]

12.2 References

- [iOS Human Interface Guidelines](#)
- [WCAG 2.1 Guidelines](#)
- [SF Symbols Browser](#)

End of Design System Specification